

How It Works

The Project

At a high level, this project is a Digital Health Twin — an AI system built to predict cardiovascular risk by looking at both the usual clinical markers and the mental health side of the picture too. So instead of relying only on things like blood pressure, cholesterol, and ECG-related inputs, it also factors in stress, anxiety, and sleep quality, which are often just as important in real-world health outcomes.

What makes it more interesting than a basic prediction model is that it does more than output a score. It trains four separate ML models — Logistic Regression, Random Forest, XGBoost, and a PyTorch neural network. On top of that, it includes a RAG pipeline built with ChromaDB and sentence-transformers, which means you can ask plain-English questions about the patient database and get grounded responses back. There is also an Insight Engine that combines the ML prediction with similar patients retrieved through RAG, turning the result into a single explainable narrative instead of a raw probability. All of it is exposed through a FastAPI backend and connected to a live web dashboard.

The Folders

Quick rundown of what lives where.

`api/` — just `main.py` in here. This runs the whole show. FastAPI server. Start this and the app is live.

`data/` — two things. `prepare_data.py` grabs the UCI Cleveland heart dataset (the classic one that researchers have been using for decades) and then tacks on synthetic mental health columns. Running it spits out `heart.csv` — 303 patients, 21 columns.

`models/` — the ML training script `train.py` plus all the saved model files it generates. You get `logistic_regression.pkl`, `random_forest.pkl`, `xgboost_model.pkl`, `pytorch_nn.pth` with its config file `pytorch_nn_config.pkl`. Also `scaler.pkl` for data normalization and `feature_cols.pkl` which is literally just the list of column names. And `metrics.json` with all the evaluation numbers.

`rag/` — the retrieval-augmented generation stuff. `embedder.py` turns patient text notes into vectors and shoves them into a ChromaDB database. `retriever.py` handles searching that database when someone asks a question. The actual database files sit in `chroma_db/`.

`insight/` — one file. `engine.py`. Honestly the most critical piece of the whole thing. Takes the ML prediction and the RAG search results and mashes them into a readable health summary.

`ui/` — `index.html`, `style.css`, `script.js`. The web interface. Three tabs. Dark theme. No frameworks.

There's also `research/` with a paper draft and `outputs/` with example API responses.

Running It

Order matters here. Cannot stress that enough. If you try to start the server before training the models, it crashes.

`pip install -r requirements.txt` — get the dependencies sorted.

`python data/prepare_data.py` — downloads the UCI data, generates synthetic mental health features, outputs `heart.csv`.

`python models/train.py` — trains four different models. Takes maybe a minute or two. Saves everything as pickle files.

`python rag/embedder.py` — vectorizes patient notes, stores them in ChromaDB.

`python api/main.py` — launches the server on port 8000. Dashboard is at <http://localhost:8000/ui/index.html>. Swagger docs at <http://localhost:8000/docs>.

That's it. Has to be in that order though.

Data — Where It Comes From

The base dataset is UCI Cleveland. Everybody in ML knows this one. 303 patients with 14 features — age, sex, chest pain type, resting BP, cholesterol, fasting blood sugar, resting ECG, max heart rate during exercise, whether exercise causes angina, ST depression, slope of the ST segment, number of major vessels from fluoroscopy, and thalassemia type. Originally the target has five classes but the script collapses them down to binary. Zero means no heart disease, one means yes. Gaps in the data get filled with medians.

Here is where things get interesting. Five extra features get generated on top of the original fourteen. Stress level on a 1-to-10 scale. Sleep quality, also 1 to 10. Weekly activity hours. An anxiety score using the GAD-7 scale which goes up to 21. And BMI. None of these come from actual patient surveys — they are synthetic. But they are not just random numbers either. There's a seed set (42, obviously) so the results are reproducible. Each synthetic feature follows a normal distribution as a baseline, and patients who actually have heart disease get their values shifted. More stress. Worse sleep. Higher anxiety. Less exercise. Everything clipped to stay in clinically reasonable ranges.

Every patient also gets a written clinical note auto-generated from their numbers. Like a mini summary. "58-year-old male, BP 145, cholesterol 260..." These notes are what the RAG system chews on later. Patient IDs run from P0000 to P0302.

Total features the ML models see: 18.

Machine Learning

The training script reads `heart.csv`, drops `patient_id` and `patient_note` (those aren't features), and does an 80/20 stratified split. A `StandardScaler` fits on the training set and gets saved — every new patient that comes in later gets normalized the exact same way.

Then it trains four models back to back.

Logistic Regression. Scikit-learn. `max_iter=1000`, default L2 penalty. Gets 95.08% accuracy, AUC of 0.988. Not bad for a linear model. Saved as `logistic_regression.pkl`.

Random Forest. 200 trees, max depth 10. Same 95.08% accuracy but AUC hits 0.994 — highest of the bunch. Pickle file is massive (914 KB) because it stores every single tree. `random_forest.pkl`.

XGBoost. 300 estimators, depth 6, learning rate 0.1. Accuracy drops to 91.80%, AUC is 0.978. Lower numbers on paper. But this is the one the API uses by default. Why? Because with only 303 samples and a 60-ish person test set, a few percentage points mean nothing statistically. XGBoost is the go-to for tabular problems in production. It generalizes better. `xgboost_model.pkl`.

PyTorch neural network. Architecture: input (18) → linear to 128 → batch norm → ReLU → dropout at 0.3 → linear to 64 → batch norm → ReLU → dropout 0.3 → linear to 32 → ReLU → linear to 1 → sigmoid. Trained 100 epochs, batch size 32, Adam optimizer with lr 0.001, BCE loss. Uses GPU if available. 93.44% accuracy, 0.987 AUC. Weights in `pytorch_nn.pth`, architecture config in `pytorch_nn_config.pkl`.

Quick results table — Logistic Regression: 95.08% acc, 93.10% precision, 96.43% recall, 94.74% F1, 0.988 AUC. Random Forest: 95.08%, 93.10%, 96.43%, 94.74%, 0.994. XGBoost: 91.80%, 87.10%, 96.43%, 91.53%, 0.978. PyTorch NN: 93.44%, 92.86%, 92.86%, 92.86%, 0.987.

The training code also tries to compute SHAP values on the XGBoost model for feature importance explanations. There was a minor bug with string conversion during our run but the implementation is there.

RAG — The Question Answering Part

This is probably the part that takes the most explaining.

RAG means Retrieval-Augmented Generation. Fancy name. Simple idea. Instead of having the system guess at answers, it first searches a database for real relevant data, and then uses that data to construct a response. Grounded answers. Nothing hallucinated.

Step one happens in `embedder.py`. It loads every patient note from the CSV. Then it runs each note through a sentence-transformer model — specifically `all-MiniLM-L6-v2`. Tiny model. 22 million parameters. Runs on CPU in under a second. What it does is turn each chunk of text into a 384-dimensional vector. A list of 384 numbers that mathematically represent the meaning of that text. All 303 vectors get stored in ChromaDB, a vector database, in a collection called "patients". Metadata goes in too — age, BMI, stress, disease label. ChromaDB persists to disk at `rag/chroma_db/` so you don't lose anything between restarts.

Step two is in `retriever.py`. Someone types a question. The retriever takes that question, runs it through the same MiniLM model to get a 384-dim vector, and asks ChromaDB to find the 5 most similar patient vectors using cosine similarity.

But then it does something clever. Before generating the response, it runs intent detection. Using regex, it categorizes the incoming question into one of ten buckets — risk, mental health, recommendations, causes, similar patients, statistics, patient-specific, demographic, lifestyle, or a general catch-all. Each bucket has trigger words. "Risk" triggers on words like danger, disease, predict. "Mental" triggers on stress, anxiety, sleep, depression. "Lifestyle" triggers on BMI, exercise, weight, diet. And so on.

Each intent maps to its own response template. The templates pull actual numbers from the retrieved patients. What percentage had heart disease. Their average age. Average BMI. Average stress level. So if 3 out of 5 matched patients had heart disease, the answer says exactly that. Real data, not generated fluff.

Worth mentioning — Flan-T5-base is loaded in the code too. It's a small language model. But the template approach turned out to produce way more reliable outputs so that is what actually gets used.

The Insight Engine

`engine.py` in the `insight/` folder. This is the piece that brings ML and RAG together.

The main function is `generate_insight()`. You pass in a patient's 18 features and it runs through several stages.

Loads the XGBoost model (falls back to Random Forest if XGBoost is missing for whatever reason). Normalizes the input with the saved scaler. Gets a probability between 0.0 and 1.0. Classifies that into Low, Moderate, High, or Critical risk.

Then it checks the raw numbers against medical thresholds. Cholesterol over 240? High severity. Between 200 and 240? Moderate. BP over 140? Hypertension, high severity. 120 to 140? Elevated, moderate. BMI over 30? Obesity, high. 25 to 30? Overweight, moderate. Stress above 7, anxiety above 14 on GAD-7, sleep below 4, activity under 2 hours per week, age over 60 — all flagged.

Simultaneously it queries ChromaDB for the 5 most similar historical patients. Computes cohort statistics — average BMI, stress, anxiety, age from those matches. Compares the current patient against those averages.

Then the narrative builder creates a paragraph out of all of this. Risk level and probability. Which factors contributed. How the patient compares to similar cases. If there are both physical and mental red flags present, it adds a sentence about that cross-domain interaction specifically.

Last step — recommendations. Every risk factor maps to a concrete suggestion. High cholesterol gets "consult physician about cholesterol management, consider statins." Severe anxiety gets "seek professional evaluation for anxiety." Sedentary lifestyle gets an exercise recommendation. And so on.

Helper functions: `predict_risk()` handles just the ML prediction. `_identify_risk_factors()` does the threshold checking. `_build_narrative()` writes the text. `_factor_phrases()` turns structured data into phrases like "elevated cholesterol (260 mg/dL)". `_mental_physical_sentence()` adds the cross-domain observation. `_generate_recommendations()` maps factors to advice.

The API Endpoints

FastAPI. Auto-generates Swagger docs. Async.

GET / — returns the API name, version, endpoints list.

GET /health — health check. Returns `{"status": "healthy"}`.

GET /model-info — reads `metrics.json` and returns evaluation metrics for all four models.

POST /predict — send 18 patient features as JSON. Gets routed to `insight.engine.predict_risk()`. Returns the prediction (0 or 1), risk label, risk level, probability, confidence, which model ran, and a list of contributing factors with severity and explanations.

POST /ask — send a query string and optional `top_k`. Goes to `rag.retriever.ask()`. Returns the original query, the generated response, source patients with similarity scores, and the count.

POST /insight — same 18 features as /predict but runs the full pipeline through `generate_insight()`. Returns the narrative text, raw ML prediction, RAG context, similar patients list, recommendations, and all risk factors.

Input validation uses Pydantic schemas. `PatientData` enforces ranges — age between 1 and 120, stress 1 to 10, anxiety 0 to 21, that kind of thing. `AskQuery` needs at least 3 characters and `top_k` between 1 and 20.

CORS is wide open (all origins). UI gets served as static files from /ui/. Server binds to 0.0.0.0:8000 via Uvicorn. Models load lazily — nothing gets pulled into memory until the first request actually needs it.

The Frontend

`index.html` — three tabs. Tab one: Risk Predictor. Form with all 18 fields. Submit it and you get a visual gauge showing risk level plus a breakdown of contributing factors. Hits `/predict`.

Tab two: Health Assistant. Chat-style interface. Type a question, get a response backed by real patient data from ChromaDB. Source patients show up as small badges. Hits `/ask`.

Tab three: Insight Engine. Four preset patient profiles baked in — a healthy 38-year-old woman, a moderate-risk 52-year-old man with borderline cholesterol, a high-risk 62-year-old man with multiple factors, and a 45-year-old woman with primarily mental health concerns. Pick one, click generate, full pipeline runs. Hits `/insight`.

`style.css` — dark theme. Background is #0a0f1e. Cards use glassmorphism — semi-transparent with backdrop blur. Purple and cyan gradient accents. Risk gauge animates. Responsive layout.

`script.js` — handles tab switching, collects form inputs into JSON, makes fetch calls to the API, renders the responses. Builds the gauge, formats chat messages, lays out insight sections.

No React. No Vue. No build step. Just HTML, CSS, and JavaScript.

Tech Stack Summary

ML: scikit-learn, XGBoost, PyTorch. Explainability: SHAP. Embeddings: sentence-transformers with all-MiniLM-L6-v2. Vector database: ChromaDB (no Docker, no cloud, just a folder). API: FastAPI with Uvicorn. Frontend: vanilla web stack. Data handling: pandas and numpy.